# X-ray project Code




Course title: Technology 2
Date: 15-01-2026
Students: Gijs Brugmans

# Table of contents

## Inhoud

1. Introduction ..... 3
2. Sprint 1 ..... 4
3. Sprint 2 ..... 8
4. Sprint 3 ..... 12
    4.1 Linked list implementation ..... 12
    4.2 Test cases ..... 21
5. Sprint 4 ..... 28
    5.1 File management ..... 29
    5.2 Menu implementation ..... 31
    5.3 Test cases ..... 34

# 1. Introduction

In this report the different segments of code that are used in the individual project will be shown. The code will always be separated in different segment to indicate in which sprint which code has been made. This way everything is clear and visible without having to scroll through the whole files filled with code.

## 2. Sprint 1

In this sprint the coding was focused on making some features for the doseAdmin file. In the following pictures it will be shown what has been made and what its purpose is.

```
#define MEASUREMENTS_SIZE       10
#define HASH_TABLE_SIZE         256
#define PATIENT_AMOUNT          100
#define MAX_PATIENT_CHARACTERS  5

typedef struct
{
    char patientName[MAX_PATIENTNAME_SIZE];
    float dose [MEASUREMENTS_SIZE];
    Date examDate [MEASUREMENTS_SIZE];
    int measurementCount[MEASUREMENTS_SIZE];
} Patient;

static Patient * hashTable[HASH_TABLE_SIZE][PATIENT_AMOUNT];
```

Picture 1: In this picture the different definitions are visible. A struct that is used for the database is made and a hashtable is created

```
static uint8_t hashFunction(char patientName[MAX_PATIENTNAME_SIZE])
{
    uint32_t hash = 0;
    for (int i = 0; i < MAX_PATIENT_CHARACTERS && patientName[i] != '\0'; i++)
    {
        hash = hash * 31 + patientName[i];
    }
    return (uint8_t)(hash % HASH_TABLE_SIZE);
}

void CreateHashTable()
{
    for (int i = 0; i < HASH_TABLE_SIZE; i++)
    {
        for (int p = 0; p < PATIENT_AMOUNT; p++)
        {
            hashTable[i][p] = NULL;
        }
    }
```

Picture 2: In this picture the hashfunction is shown and how the data gets its address and the hashtable is created

```c
void RemoveAllDataFromHashTable()
{
    for (int i = 0; i < HASH_TABLE_SIZE; i++)
    {
        for (int p = 0; p < PATIENT_AMOUNT; p++)
        {
            if (hashTable[i][p] != NULL)
            {
                free(hashTable[i][p]);
                hashTable[i][p] = NULL;
            }
        }
    }
}
```

Picture 3: In this picture it is shown how the hashtable gets cleared of all its data

```c
int8_t AddPatient(char patientName[MAX_PATIENTNAME_SIZE])
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -3;
    }

    uint8_t hash = hashFunction(patientName);

    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        if (hashTable[hash][i] != NULL &&
            strncmp(hashTable[hash][i]->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            return -1;
        }
    }

    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        if (hashTable[hash][i] == NULL)
        {
            Patient* newPatient = (Patient*)malloc(sizeof(Patient));
            if (!newPatient)
            {
                return -2;
            }
            memset(newPatient, 0, sizeof(Patient));
            strncpy(newPatient->patientName, patientName, MAX_PATIENTNAME_SIZE - 1);
            newPatient->patientName[MAX_PATIENTNAME_SIZE - 1] = '\0';
            hashTable[hash][i] = newPatient;
            return 0;
        }
    }
    return -2;
}
```

Picture 4: In this picture it is shown how a new patient gets registered into the hash table and which checks it has to go through to accomplish this command

```c
int8_t RemovePatient(char patientName[MAX_PATIENTNAME_SIZE])
{
    // Check string length
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);
    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        if (hashTable[hash][i] != NULL &&
            strncmp(hashTable[hash][i]->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            free(hashTable[hash][i]);
            hashTable[hash][i] = NULL;
            return 0;
        }
    }
    return -1;
}
```

Picture 5: In this picture it is shown how a patient is removed from the hashtable and which checks it has to go through to accomplish this command.

```c
int8_t IsPatientPresent(char patientName[MAX_PATIENTNAME_SIZE])
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);

    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        if (hashTable[hash][i] != NULL &&
            strncmp(hashTable[hash][i]->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            return 0;
        }
    }
    return -1;
}
```

Picture 6: In this picture the code is shown on how it is checked if a patient is already present in the database.

# 3. Sprint 2

In this sprint the coding was also focused on the doseAdmin file. In the following pictures it is visible what was made during this sprint.

```
int8_t AddPatientDose(char patientName[MAX_PATIENTNAME_SIZE],
                      Date* date, uint16_t dose)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -3;
    }

    uint8_t hash = hashFunction(patientName);
    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        Patient *patient = hashTable[hash][i];
        if (patient != NULL &&
            strncmp(patient->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
            {
                if (patient->measurementCount[measure] == 0)
                {
                    patient->dose[measure] = (float)dose;
                    patient->measurementCount[measure] = 1;
                    patient->examDate[measure] = *date;
                    return 0;
                }
            }
            return -2;
        }
    }
    return -1;
}
```

Picture 7: In this picture it is visible how the dosage of a patient is added to the database and the multiple dose data variables get added to the file of a specific patient

```c
// Converting a Date to an integer YYYMMDD for easy comparison
static inline uint32_t dateToKey(const Date* dateInt)
    {
        return (uint32_t)(dateInt->year) * 10000u + (uint32_t)(dateInt->month) * 100u + (uint32_t)(dateInt->day);
    };

int8_t PatientDoseInPeriod(char patientName[MAX_PATIENTNAME_SIZE],
                           Date* startDate, Date* endDate, uint32_t* totalDose)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }
    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint32_t startKey = dateToKey(startDate);
    uint32_t endKey   = dateToKey(endDate);

    uint8_t hash = hashFunction(patientName);

    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        Patient *p = hashTable[hash][i];
        if (p != NULL &&
            strncmp(p->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            uint32_t sum = 0u;

            for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
            {
                if (p->measurementCount[measure] == 0)
                {
                    continue;
                }
                uint32_t examKey = dateToKey(&p->examDate[measure]);
                if (examKey >= startKey && examKey <= endKey)
                {
                    sum += (uint32_t)(p->dose[measure]);
                }
            }

            if (totalDose != NULL)
            {
                *totalDose = sum;
            }
            return 0;
        }
    }
    return -1;
}
```

Picture 8: In this picture it is visible how the sum of the total dosage a patient accumulated over a period of time is calculated. The date of the period first gets converted to an integer for an easier comparison.

```c
int8_t GetNumberOfMeasurements(char patientName[MAX_PATIENTNAME_SIZE],
                               size_t * nrOfMeasurements)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }
    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);

    for (int i = 0; i < PATIENT_AMOUNT; i++)
    {
        Patient *patient = hashTable[hash][i];
        if (patient != NULL &&
            strncmp(patient->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
        {
            size_t count = 0;
            for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
            {
                if (patient->measurementCount[measure] != 0)
                {
                    count++;
                }
            }
            if (nrOfMeasurements != NULL)
            {
                *nrOfMeasurements = count;
            }
            return 0;
        }
    }

    return -1;
}
```

Picture 9: In this picture it is shown how the program knows how many measurements a specific patient has had.

```c
void GetHashPerformance(size_t *totalNumberOfPatients, double *averageNumberOfPatients,
                        double *standardDeviation)
{
    size_t total = 0;
    size_t counts[HASH_TABLE_SIZE];

    for (size_t i = 0; i < HASH_TABLE_SIZE; i++)
    {
        size_t count = 0;
        for (size_t p = 0; p < PATIENT_AMOUNT; p++)
        {
            if (hashTable[i][p] != NULL)
            {
                count++;
            }
        }
        counts[i] = count;
        total += count;
    }

    if (totalNumberOfPatients != NULL)
    {
        *totalNumberOfPatients = total;
    }

    double avg = 0.0;
    if (HASH_TABLE_SIZE > 0)
    {
        avg = (double)total / (double)HASH_TABLE_SIZE;
    }

    if (averageNumberOfPatients != NULL)
    {
        *averageNumberOfPatients = avg;
    }

    double sum = 0.0;
    for (size_t i = 0; i < HASH_TABLE_SIZE; i++)
    {
        double diff = (double)counts[i] - avg;
        sum += diff * diff;
    }

    double sdev = 0.0;

    if (HASH_TABLE_SIZE > 0)
    {
        sdev = sqrt(sum / (double)HASH_TABLE_SIZE);
    }

    if (standardDeviation != NULL)
    {
        *standardDeviation = sdev;
    }
}
```

Pictures 10 & 11: In these pictures it is shown how the performance of the hashtable gets checked. This function checks if the hashtable is good enough.

# 4. Sprint 3

In this sprint a linked list had to be implemented into the DoseAdmin. Other than that every function of the DoseAdmin got test cases testing different outcomes and return values.

## 4.1 Linked list implementation

In this part of the report every function within the DoseAdmin is shown that got the linked list implementation. A linked list gives all the data in a database an adres and with the help of a pointer all the data gets linked to each other.

```
typedef struct Patient
{
    char patientName[MAX_PATIENTNAME_SIZE];
    float dose [MEASUREMENTS_SIZE];
    Date examDate [MEASUREMENTS_SIZE];
    int measurementCount[MEASUREMENTS_SIZE];
    Patient *next;
} Patient;
```

Picture 12: In this picture above the Patient struct is visible with the only thing being changed is a pointer that points to the adres of the next Patient in the dataset

```
void CreateHashTable()
{
    for (int i = 0; i < HASH_TABLE_SIZE; i++)
    {
        hashTable[i] = NULL;
    }
}
```

Picture 13: In the picture above the new CreateHashTable function is shown, it is a lot more simplified because there is no limit to the amount of patients that can be saved

```
void RemoveAllDataFromHashTable()
{
    for (int i = 0; i < HASH_TABLE_SIZE; i++)
    {
        Patient *current = hashTable[i];
        while (current != NULL)
        {
            Patient *next = current->next;
            free(current);
            current = next;
        }
        hashTable[i] = NULL;
    }
}
```

Picture 14: In the picture above the RemoveAllDataFromHashTable is shown. It goes through the entire string of addresses to delete all the data in them.

```c
int8_t AddPatient(char patientName[MAX_PATIENTNAME_SIZE])
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -3;
    }

    uint8_t hash = hashFunction(patientName);

    for (Patient *current = hashTable[hash]; current != NULL; current = current->next)
    {
        if (strncmp(current->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
            return -1;
    }

    // patient does not exist in the hashastable, create new
    Patient *newPatient = calloc(1, sizeof(Patient));
    if (!newPatient)
    {
        return -2;
    }

    strncpy(newPatient->patientName, patientName, MAX_PATIENTNAME_SIZE - 1);
    newPatient->patientName[MAX_PATIENTNAME_SIZE - 1] = '\0';
    newPatient->next = hashTable[hash];
    hashTable[hash] = newPatient;
    return 0;
}
```

Picture 15: In the picture above the AddPatient function is shown. It now adds the new patient to the front of the list, and gives it a pointer that points to the patient that before this patient was the first one on the list

```c
int8_t AddPatientDose(char patientName[MAX_PATIENTNAME_SIZE],
                      Date* date, uint16_t dose)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -3;
    }

    uint8_t hash = hashFunction(patientName);
    Patient *patient = hashTable[hash];
    while (patient != NULL && strncmp(patient->patientName, patientName, MAX_PATIENTNAME_SIZE) != 0)
    {
        patient = patient->next;
    }

    if (patient == NULL)
    {
        return -1;
    }

    for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
    {
        if (patient->measurementCount[measure] == 0)
        {
            patient->dose[measure] = (float)dose;
            patient->measurementCount[measure] = 1;
            patient->examDate[measure] = *date;
            return 0;
        }
    }
    return -2;
}
```

Picture 16: In the picture above the AddPatientDose function is shown. It now searches through the database until it finds the correct patient and than adds the dose they got from the measurement

```c
int8_t PatientDoseInPeriod(char patientName[MAX_PATIENTNAME_SIZE],
                           Date* startDate, Date* endDate, uint32_t* totalDose)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint32_t startKey = dateToKey(startDate);
    uint32_t endKey   = dateToKey(endDate);

    uint8_t hash = hashFunction(patientName);

    Patient *p = hashTable[hash];
    while (p != NULL && strncmp(p->patientName, patientName, MAX_PATIENTNAME_SIZE) != 0)
    {
        p = p->next;
    }

    if (p == NULL)
    {
        return -1;
    }

    uint32_t sum = 0u;
    for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
    {
        if (p->measurementCount[measure] == 0)
        {
            continue;
        }

        uint32_t examKey = dateToKey(&p->examDate[measure]);
        if (examKey >= startKey && examKey <= endKey)
        {
            sum += (uint32_t)(p->dose[measure]);
        }
    }

    if (totalDose != NULL)
    {
        *totalDose = sum;
    }

    return 0;
}
```

Picture 17: In the picture above the PatientDoseInPeriod is shown. The only thing that changed from the previous code is that the way the patient is searched is now done by pointing the the next pointer, that makes sure that they go to the next patient in the database

```c
int8_t RemovePatient(char patientName[MAX_PATIENTNAME_SIZE])
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);
    Patient *previous = NULL;
    Patient *current = hashTable[hash];

    while (current != NULL && strncmp(current->patientName, patientName, MAX_PATIENTNAME_SIZE) != 0)
    {
        previous = current;
        current = current->next;
    }

    if (current == NULL)
    {
        return -1;
    }

    if (previous == NULL)
    {
        hashTable[hash] = current->next;
    }

    else
    {
        previous->next = current->next;
    }

    free(current);
    return 0;
}
```

Picture 18: In the picture above the RemovePatient function is shown. It now deletes the patient, and replaces the next pointer so that the list points to the next patient that still exists within the database.

```c
int8_t IsPatientPresent(char patientName[MAX_PATIENTNAME_SIZE])
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);
    for (Patient *current = hashTable[hash]; current != NULL; current = current->next)
    {
        if (strncmp(current->patientName, patientName, MAX_PATIENTNAME_SIZE) == 0)
            return 0;
    }
    return -1;
}
```

Picture 19: In the picture above the IsPatientPresent function is shown. This function now searches through the entire database with the help of the next pointer until it either finds the correct patient or doesn't find it and returns that.

```c
int8_t GetNumberOfMeasurements(char patientName[MAX_PATIENTNAME_SIZE],
                               size_t * nrOfMeasurements)
{
    size_t length = 0;
    while (patientName[length] != '\0' && length < MAX_PATIENTNAME_SIZE)
    {
        length++;
    }

    if (length >= MAX_PATIENTNAME_SIZE && patientName[length] != '\0')
    {
        return -2;
    }

    uint8_t hash = hashFunction(patientName);
    Patient *patient = hashTable[hash];
    while (patient != NULL && strncmp(patient->patientName, patientName, MAX_PATIENTNAME_SIZE) != 0)
    {
        patient = patient->next;
    }

    if (patient == NULL)
    {
        return -1;
    }

    size_t count = 0;
    for (int measure = 0; measure < MEASUREMENTS_SIZE; measure++)
    {
        if (patient->measurementCount[measure] != 0)
        {
            count++;
        }
    }

    if (nrOfMeasurements != NULL)
    {
        *nrOfMeasurements = count;
    }
    return 0;
}
```

Picture 20: In the picture above the GetNumberOfMeasurements function is shown. This function now uses the linked list to search for the patient and the measurements connected to that patient.

```c
void GetHashPerformance(size_t *totalNumberOfPatients, double *averageNumberOfPatients,
                        double *standardDeviation)
{
    size_t total = 0;
    size_t counts[HASH_TABLE_SIZE];

    for (size_t i = 0; i < HASH_TABLE_SIZE; i++)
    {
        size_t count = 0;
        for (Patient *it = hashTable[i]; it != NULL; it = it->next)
        {
            count++;
        }
        counts[i] = count;
        total += count;
    }

    if (totalNumberOfPatients != NULL)
    {
        *totalNumberOfPatients = total;
    }

    double avg = 0.0;
    if (HASH_TABLE_SIZE > 0)
    {
        avg = (double)total / (double)HASH_TABLE_SIZE;
    }

    if (averageNumberOfPatients != NULL)
    {
        *averageNumberOfPatients = avg;
    }

    double sum = 0.0;
    for (size_t i = 0; i < HASH_TABLE_SIZE; i++)
    {
        double diff = (double)counts[i] - avg;
        sum += diff * diff;
    }

    double sdev = 0.0;

    if (HASH_TABLE_SIZE > 0)
    {
        sdev = sqrt(sum / (double)HASH_TABLE_SIZE);
    }

    if (standardDeviation != NULL)
    {
        *standardDeviation = sdev;
    }
}
```

Picture 21: In the picture above the GetHashPerformance is shown. The only thing that is changed is that there isn't a patient limit anymore, so the way the amount of patients stored in the hashtable were counted had to be changed

## 4.2 Test cases

In this part of the report the different test cases that were made this sprint will be shown. Test cases are supposed to test a part of the code by giving the part of the code that will be tested a forced set of data and testing if the predicted outcome will be the outcome that actually happens.

```
int main(void)
{
    UnityBegin();

    MY_RUN_TEST(test_AddingPatientSuccesfully);
    MY_RUN_TEST(test_AddingPatientWithDuplicatePatient);
    MY_RUN_TEST(test_AddingPatientButTooLongName);
    MY_RUN_TEST(test_AddingPatientDoseToUnknownPatient);
    MY_RUN_TEST(test_AddingPatientDoseSuccesfully);
    MY_RUN_TEST(test_AddingPatientDoseFailingAllocatingMemory);
    MY_RUN_TEST(test_CalculatigPatientDoseInPeriodSuccesfully);
    MY_RUN_TEST(test_CalculatingPatientDoseInPeriodToUnknownPatient);
    MY_RUN_TEST(test_CalculatingPatientDoseInPeriodButNameTooLong);
    MY_RUN_TEST(test_RemovingPatientsuccesfully);
    MY_RUN_TEST(test_RemovingUnknownPatient);
    MY_RUN_TEST(test_RemovingPatientWithTooLongName);
    MY_RUN_TEST(test_PatientIsPresent);
    MY_RUN_TEST(test_PatientIsNotPresent);
    MY_RUN_TEST(test_IsPatientPresentWithTooLongName);
    MY_RUN_TEST(test_GetNumberOfMeasurementsSuccesfully);
    MY_RUN_TEST(test_GetNumberOfMeasurementsOfUnknownPatient);
    MY_RUN_TEST(test_GetNumberOfMeasurementsWithTooLongName);

    UnityEnd();
    return 0;
}
```

Picture 22: in the picture above the all the different test cases are shown.

```
static void create_too_long_name(char *buffer, size_t buffersize)
{
    memset(buffer, 'X', MAX_PATIENTNAME_SIZE + 1);
}
```

Picture 23: In the picture above the help function create_too_long_name is shown. This function is used throughout the testcases and just makes a memory set that contains a string of x's 1 character longer than the MAX_PATIENTNAME_SIZE

```c
void test_AddingPatientSuccesfully(void)
{
    char name[] = "John";
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
}

void test_AddingPatientWithDuplicatePatient(void)
{
    char name[] = "John";
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    TEST_ASSERT_EQUAL_INT(-1, AddPatient(name));
}

void test_AddingPatientButTooLongName(void)
{
    char longname[MAX_PATIENTNAME_SIZE + 1];
    create_too_long_name(longname, MAX_PATIENTNAME_SIZE + 1);
    TEST_ASSERT_EQUAL_INT(-3, AddPatient(longname));
}
```

Picture 24: In the picture abouve the test cases for the AddPatient function are shown. The first adds a patient succesfully, the second triest o add a duplicate patient and the third tries to add a patient with a name that is too long

```c
void test_AddingPatientDoseToUnknownPatient(void)
{
    Date d = {.day = 4, .month = 11, .year = 2025};
    TEST_ASSERT_EQUAL_INT(-1, AddPatientDose("John", &d, 100));
}

void test_AddingPatientDoseSuccesfully(void)
{
    char name[] = "John";
    Date d = {.day = 11, .month = 11, .year = 2025};
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    for (int i = 0; i < MEASUREMENTS_SIZE; i++)
    {
        TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &d, (uint16_t)(10 + i)));
    }
}

void test_AddingPatientDoseFailingAllocatingMemory(void)
{
    char name[] = "John";
    Date d = {.day = 11, .month = 11, .year = 2025};
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    for (int i = 0; i < MEASUREMENTS_SIZE; i++)
    {
        TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &d, (uint16_t)(10 + i)));
    }
    TEST_ASSERT_EQUAL_INT(-2, AddPatientDose(name, &d, 999));
}
```

Picture 25: In the picture above the test cases for the AddPatientDose function are shown. The first test case tries to add a dose to an unknown patient, the second test case adds the dose to the patient succesfully and the third test case fails to allocate the memory of the dose to the patient

```c
void test_CalculatigPatientDoseInPeriodSuccesfully(void)
{
    char name[] = "John";
    Date measurement1 = {.day = 1, .month = 1, .year = 2025};
    Date measurement2 = {.day = 30, .month = 1, .year = 2025};
    Date start = {.day = 1, .month = 1, .year = 2025};
    Date end   = {.day = 31, .month = 1, .year = 2025};
    uint32_t total = 0;

    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &measurement1, 50));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &measurement2, 75));

    TEST_ASSERT_EQUAL_INT(0, PatientDoseInPeriod(name, &start, &end, &total));
    TEST_ASSERT_EQUAL_UINT32(125u, total);
}

void test_CalculatingPatientDoseInPeriodToUnknownPatient(void)
{
    Date start = {.day = 1, .month = 1, .year = 2025};
    Date end = {.day = 31, .month = 12, .year = 2025};
    uint32_t total = 0;
    TEST_ASSERT_EQUAL_INT(-1, PatientDoseInPeriod("Blank", &start, &end, &total));
}

void test_CalculatingPatientDoseInPeriodButNameTooLong(void)
{
    Date start = {.day = 1, .month = 1, .year = 2025};
    Date end = {.day = 31, .month = 12, .year = 2025};
    uint32_t total = 0;
    char *longname = malloc(MAX_PATIENTNAME_SIZE + 1);
    create_too_long_name(longname, MAX_PATIENTNAME_SIZE + 1);
    TEST_ASSERT_EQUAL_INT(-2, PatientDoseInPeriod(longname, &start, &end, &total));
    free(longname);
}
```

Picture 26: In the picture above the test cases for the CalculatePatientDoseInPeriod function are shown. The first test case calculates the dose of the patient succesfully, the second test case tries to calculate the dose of an unknown patient and the third test case tries to calculate the dose of a patient with a name that is too long.

```
void test_RemovingPatientsuccesfully(void)
{
    char name[] = "John";
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    TEST_ASSERT_EQUAL_INT(0, RemovePatient(name));
}

void test_RemovingUnknownPatient(void)
{
    char name[] = "John";
    TEST_ASSERT_EQUAL_INT(-1, RemovePatient(name));
}

void test_RemovingPatientWithTooLongName(void)
{
    char *longname = malloc(MAX_PATIENTNAME_SIZE + 1);
    create_too_long_name(longname, MAX_PATIENTNAME_SIZE + 1);
    TEST_ASSERT_EQUAL_INT(-2, RemovePatient(longname));
    free(longname);
}
```

Picture 27: In the picture above the test cases for the RemovePatient function are shown. The first test case removes a patient succesfully, the second test case tries to remove a patient that doesn't in the database and the third test case tries to remove a patient that has a name that is too long.

```
void test_PatientIsPresent(void)
{
    char name[] = "John";
    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    TEST_ASSERT_EQUAL_INT(0, IsPatientPresent(name));
}

void test_PatientIsNotPresent(void)
{
    TEST_ASSERT_EQUAL_INT(-1, IsPatientPresent("Blank"));
}

void test_IsPatientPresentWithTooLongName(void)
{
    char *longname = malloc(MAX_PATIENTNAME_SIZE + 1);
    create_too_long_name(longname, MAX_PATIENTNAME_SIZE + 1);
    TEST_ASSERT_EQUAL_INT(-2, IsPatientPresent(longname));
    free(longname);
}
```

Picture 28: In the picture above the test cases for the IsPatientPresent function are shown. The first test case finds that the patient is present in the database, the second test case searches for a patient name that isn't present in the database and the third test case searches for a patient name that is too long

```c
void test_GetNumberOfMeasurementsSuccesfully(void)
{
    char name[] = "John";
    Date measurement = {.day = 27, .month = 11, .year = 2025};
    size_t n = 0;

    TEST_ASSERT_EQUAL_INT(0, AddPatient(name));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &measurement, 10));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(name, &measurement, 20));
    TEST_ASSERT_EQUAL_INT(0, GetNumberOfMeasurements(name, &n));
    TEST_ASSERT_EQUAL_UINT32(2u, (uint32_t)n);
}

void test_GetNumberOfMeasurementsOfUnknownPatient(void)
{
    size_t n = 0;
    TEST_ASSERT_EQUAL_INT(-1, GetNumberOfMeasurements("Blank", &n));
}

void test_GetNumberOfMeasurementsWithTooLongName(void)
{
    char *longname = malloc(MAX_PATIENTNAME_SIZE + 1);
    create_too_long_name(longname, MAX_PATIENTNAME_SIZE + 1);
    size_t nr = 0;
    TEST_ASSERT_EQUAL_INT(-2, GetNumberOfMeasurements(longname, &nr));
    free(longname);
}
```

Picture 29: In the picture above the test cases for the GetNumberOfMeasurements function are shown. The first test case gets the number of measurements of a specific patient successfully, the second test case tries to get the number of measurements of a patient that doesn't exist in the database and the third test case tries to get the number of measurements of a patient that has a name that is too long

# 5. Sprint 4

In this sprint there are a couple of things that happened to the individual project. First of all a file management system got implemented so that patients that are saved in the database don't get lost when the machine gets turned off. Furthermore the menu now works when the machine is active and lastly some testcases where implemented to test the function for the file management

## 5.1 File management

In this part of the report the WriteToFile and ReadFromFile functions will be shown

```
int8_t WriteToFile(char filePath[MAX_FILEPATH_LEGTH])
{
    if (filePath == NULL) //ok
    {
        return -1;
    }

    FILE* PatientData = fopen(filePath, "w"); // ok : you choose a text file
    if (PatientData == NULL) // ok
    {
        return -1;
    }
int status = 0;

for (size_t pd = 0; pd < HASH_TABLE_SIZE && status == 0; pd++) // ok
{
    Patient *p = hashTable[pd]; //ok
    while (p != NULL && status == 0)
    {
        // Count measurements for this patient
        int measurementCount = 0;
        for (int m = 0; m < MEASUREMENTS_SIZE; m++)
        {
            if (p->measurementCount[m] != 0)
                measurementCount++;
        }

        // Write patientName
        if (fprintf(PatientData, "Patient:%s\n", p->patientName) < 0)
        {
            status = -1;
            break;
        }

        // Write number of measurements
        if (fprintf(PatientData, "Measurements:%d\n", measurementCount) < 0)
        {
            status = -1;
            break;
        }

        // Write each measurement as: day month year dose (dose printed as float)
        for (int m = 0; m < MEASUREMENTS_SIZE; m++)
        {
            if (p->measurementCount[m] != 0)
            {
                if (fprintf(PatientData, "%u %u %u %.2f\n",
                            (unsigned int)p->examDate[m].day,
                            (unsigned int)p->examDate[m].month,
                            (unsigned int)p->examDate[m].year,
                            p->dose[m]) < 0)
                {
                    status = -1;
                    break;
                }
            }
        }

        p = p->next;
    }
}

status = fclose(PatientData);

// 0 = ok, -1 = error
return (status);
```

Pictures 30 & 31: In these two picture shown above the code for the WriteToFile function. This function gets activated whenever the system gets turned off and writes all the data that is stored in the database to a txt file that gets stored locally.

```c
int8_t ReadFromFile(char filePath[MAX_FILEPATH_LEGTH])
{
    if (filePath == NULL)
    {
        return -1;
    }

    FILE* PatientData = fopen(filePath, "r");
    if (PatientData == NULL)
    {
        return -1;
    }
int status = 0;
char patientName[MAX_PATIENTNAME_SIZE];
int measurementCount = 0;

while (status == 0)
{
    // Try to read "Patient:name" line.
    int scanResult = fscanf(PatientData, " Patient:%319s", patientName);
    if (scanResult == EOF)
    {
        break;
    }
    if (scanResult != 1)
    {
        status = -1;
        break;
    }

    // Add patient to hash table
    int8_t result = AddPatient(patientName);
    if (result != 0 && result != -1)
    {
        status = -1;
        break;
    }
    }

    // Read Measurementscount
    if (fscanf(PatientData, " Measurements:%d", &measurementCount) != 1)
    {
        status = -1;
        break;
    }

    // Read measurement lines
    for (int i = 0; i < measurementCount && status == 0; i++)
    {
        uint8_t day;
        uint8_t month;
        uint16_t year;
        float dose;

        if (fscanf(PatientData, " %hhu %hhu %hu %f", &day, &month, &year, &dose) != 4)
        {
            status = -1;
            break;
        }

        Date date = {day, month, year};
        int8_t result = AddPatientDose(patientName, &date, (uint16_t)dose);
        if (result != 0)
        {
            status = -1;
            break;
        }
    }
}

if (fclose(PatientData) == EOF)
{
    status = -1;
}

return (status);
```

Picture 32 & 33: In the two pictures show above the ReadFromFile is shown. This function gets activated whenever the machine starts up. It gets access to the txt file that was created from the WriteToFile function and read all the data and puts it back into the database.

## 5.2 Menu implementation

The menu that is used inside the individual project has 5 options to be programmed. These 5 options will be shown below.

```
int main(int argc, char* argv[])
{
    static CENTRAL_ACQUISITION_CONNECTION_STATE centralAcqConnectionState = NOT_CONNECTED_WITH_CENTRAL_ACQUISITION;
    if (connectWithCentralAcquisition()) {
        centralAcqConnectionState = CONNECTED_WITH_CENTRAL_ACQUISITION;
    }
    else {
        printf("\n\nConnecting with CentralAcquisition Failed. No problem, you can continue with \n");
        printf("the functionality that does not depend on that connection!\n");
    }

    fcntl(0, F_SETFL, fcntl(0, F_GETFL) | O_NONBLOCK);
    char selectedPatient[MAX_PATIENTNAME_SIZE] = "";

    // Initialize dose admin and try to read persisted data
    CreateHashTable();
    const char standardPatientName[] = "JohnDoe";
    SetReservedPatientName(standardPatientName);
    AddPatient((char*)standardPatientName);
    const char dataFile[] = "patients.txt";
    if (ReadFromFile((char*)dataFile) == 0) {
        printf("Loaded patient data from %s\n", dataFile);
    }
```

Picture 34: In the picture shown above the startup is shown of the project. The text file gets loaded read so that any patients who were stored in the text file get put back into the database.

```
case MO_ADD_PATIENT:
    {
        char patientName[MAX_PATIENTNAME_SIZE];
        printf("Enter patient name: ");
        {
            char bufferLine[BUFFER_BYTE];
            if (getLineFromUser(bufferLine, sizeof(bufferLine)) == 0) {
                if (sscanf(bufferLine, "%319s", patientName) == 1) {
                    int8_t r = AddPatient(patientName);
                    if (r == 0)
                    {
                        printf("Patient '%s' added.\n", patientName);
                    }
                    else
                    {
                        printf("Failed to add patient into the database(code %d).\n", r);
                    }
                }
            }
        }

    }
    break;
```

Picture 35: In the picture shown above the execution for when add patient is chosen is shown. After a patient name is entered by the admin it gets a spot in the linked list and also a connection to the previous patient that was added into the linked list to it stays linked.

```c
case MO_DELETE_PATIENT:
    {
        char patientName[MAX_PATIENTNAME_SIZE];
        printf("Enter patient name to delete: ");
        {
            char bufferLine[BUFFER_BYTE];
            if (getLineFromUser(bufferLine, sizeof(bufferLine)) == 0) {
                if (sscanf(bufferLine, "%319s", patientName) == 1) {
                    if (strncmp(patientName, standardPatientName, MAX_PATIENTNAME_SIZE) == 0)
                    {
                        printf("Cannot remove this patient '%s'.\n", standardPatientName);
                    }
                    else
                    {
                        int8_t r = RemovePatient(patientName);
                        if (r == 0)
                        {
                            printf("Patient '%s' removed.\n", patientName);
                        }
                        else
                        {
                            printf("Failed to remove patient out of database (code %d).\n", r);
                        }
                    }
                }
            }
        }
    }
    break;
```

Picture 36: In the picture shown above the execution of when a patient gets deleted is shown. Whenever a patient name is entered by the admin the RemovePatient function is triggered and based on the return value the monitor returns a different text

```c
case MO_SELECT_PATIENT:
    {
        char patientName[MAX_PATIENTNAME_SIZE];
        printf("Enter patient name to select: ");
        {
            char bufferLine[BUFFER_BYTE];
            if (getLineFromUser(bufferLine, sizeof(bufferLine)) == 0) {
                if (sscanf(bufferLine, "%319s", patientName) == 1) {
                    int8_t r = IsPatientPresent(patientName);
                    if (r == 0)
                    {
                        size_t nr = 0;
                        GetNumberOfMeasurements(patientName, &nr);
                        printf("Patient '%s' is present with %zu measurements.\n", patientName, nr);
                        strncpy(selectedPatient, patientName, MAX_PATIENTNAME_SIZE - 1);
                        selectedPatient[MAX_PATIENTNAME_SIZE - 1] = '\0';
                    }
                    else
                    {
                        printf("Patient '%s' not found inside database (code %d).\n", patientName, r);
                    }
                }
            }
        }
    }
    break;
```

Picture 37: In the picture shown above the execution of when a patient gets selected is shown. Whenever a patient name is entered by the admin the system first checks if the patient is present inside the database, after that it either triggers GetNumberOfMeasurements and returns that or it returns that the patient is not found

```
case MO_QUIT:
    disconnectFromCentralAcquisition();
    centralAcqConnectionState = NOT_CONNECTED_WITH_CENTRAL_ACQUISITION;
    if (WriteToFile((char*)dataFile) == 0)
    {
        printf("Saved patient data to %s\n", dataFile);
    }
    else
    {
        printf("Failed to save patient data to %s\n", dataFile);
    }
    return 0;
    break;
default:
    printf("Please, enter a valid number! %d\n", choice);
    break;
```

Picture 38: In the picture shown above the execution of quitting the system is shown. Whenever this gets triggered the WriteToFile is triggered to get everything that is inside the linked list inside a text file to be stored locally and after that stop the system of working. Below that is the trigger for if something else is selected that doesn't trigger one of the functions that have been shown in this report

## 5.3 Test cases

In this part of the report the different test cases that were made this sprint will be shown for the WriteToFile and ReadFromFile functions.

```
static char dbPatientName[] = "Brice";
static char dbPatientName2[] = "Koen";

void setUp(void)
{
    CreateHashTable();
    AddPatient(dbPatientName);
    AddPatient(dbPatientName2);

    FILE *readingtest1 = fopen("ReadingTest.txt", "w");
    if (readingtest1 != NULL)
    {
        fprintf(readingtest1, "Patient:Brice\n");
        fprintf(readingtest1, "Measurements:1\n");
        fprintf(readingtest1, "3 7 2025 5.00\n");
        fclose(readingtest1);
    }
}

void tearDown(void)
{
    // This is run after EACH test
    // Brice: clean your patient table
    RemoveAllDataFromHashTable();
    remove("ReadingTest.txt");
}
```

Picture 39: In the picture above the setup of the testcases is shown, this was needed to make the testcases easier to maintain.

```c
void test_WriteToFileSuccesfully(void)
{
    Date d = {.day = 15, .month = 6, .year = 2025};
    AddPatientDose(dbPatientName, &d, 50);

    TEST_ASSERT_EQUAL_INT(0, WriteToFile("TestFile.txt"));
}

void test_WriteToFileWithPathBeingNULL(void)
{
    TEST_ASSERT_EQUAL_INT(-1, WriteToFile(NULL));
}

void test_WriteToFileWithUnknownPath(void)
{
    TEST_ASSERT_EQUAL_INT(-1, WriteToFile("/Unknown/TestFile.txt"));
}
```

Picture 40: In the picture above the testcases for the WriteToFile function are shown. In the first test the data gets written to the file successfully, in the second test it is tries to write something to a file with a path being NULL and the last test tries to write something to a file with an unknown path.

```c
void test_ReadFromFileSuccesfully(void)
{
    RemoveAllDataFromHashTable();

    TEST_ASSERT_EQUAL_INT(0, ReadFromFile("ReadingTest.txt"));
    TEST_ASSERT_EQUAL_INT(0, IsPatientPresent("Brice"));
    size_t n = 0;
    TEST_ASSERT_EQUAL_INT(0, GetNumberOfMeasurements("Brice", &n));
    TEST_ASSERT_EQUAL_UINT32(1u, (uint32_t)n);
}

void test_ReadFromFileWithPathBeingNULL(void)
{
    TEST_ASSERT_EQUAL_INT(-1, ReadFromFile(NULL));
}

void test_ReadFromFileWithUnknownPath(void)
{
    TEST_ASSERT_EQUAL_INT(-1, ReadFromFile("/Unknown/ReadingTest.txt"));
}
```

Picture 41: In the picture above the testcases fort he ReadFromFile function are shown. In the first test the data from the text file get read successfully, in the second test it is tries to read a file with the path being NULL and the last test tries to read a file from which the path is unknown.

```c
void intergrationTest_WriteThenReadCorrectly(void)
{
    Date measurement1 = {.day = 30, .month = 7, .year = 2025};
    Date measurement2 = {.day = 10, .month = 9, .year = 2025};
    Date measurement3 = {.day = 24, .month = 2, .year = 2025};

    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(dbPatientName, &measurement1, 50));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(dbPatientName, &measurement2, 25));
    TEST_ASSERT_EQUAL_INT(0, AddPatientDose(dbPatientName2, &measurement3, 18));
    TEST_ASSERT_EQUAL_INT(0, WriteToFile("IntegrationTest.txt"));

    RemoveAllDataFromHashTable();
    CreateHashTable();

    TEST_ASSERT_EQUAL_INT(0, ReadFromFile("IntegrationTest.txt"));
    TEST_ASSERT_EQUAL_INT(0, IsPatientPresent(dbPatientName));
    TEST_ASSERT_EQUAL_INT(0, IsPatientPresent(dbPatientName2));

    size_t n = 0;
    TEST_ASSERT_EQUAL_INT(0, GetNumberOfMeasurements(dbPatientName, &n));
    TEST_ASSERT_EQUAL_UINT32(2u, (uint32_t)n);

    size_t n2 = 0;
    TEST_ASSERT_EQUAL_INT(0, GetNumberOfMeasurements(dbPatientName2, &n2));
    TEST_ASSERT_EQUAL_UINT32(1u, (uint32_t)n2);

    // Check each measurement is present with correct dose
    uint32_t doseOnDateCheck = 0;

    TEST_ASSERT_EQUAL_INT(0, PatientDoseInPeriod(dbPatientName, &measurement1, &measurement1, &doseOnDateCheck));
    TEST_ASSERT_EQUAL_UINT32(50u, doseOnDateCheck);

    TEST_ASSERT_EQUAL_INT(0, PatientDoseInPeriod(dbPatientName, &measurement2, &measurement2, &doseOnDateCheck));
    TEST_ASSERT_EQUAL_UINT32(25u, doseOnDateCheck);

    TEST_ASSERT_EQUAL_INT(0, PatientDoseInPeriod(dbPatientName2, &measurement3, &measurement3, &doseOnDateCheck));
    TEST_ASSERT_EQUAL_UINT32(18u, doseOnDateCheck);

    remove("IntegrationTest.txt");
}
```

Picture 42: In the picture above a integration test is shown where the WriteToFile and ReadFromFile have to both end correctly. It first writes up a file with one patient that contains two measurements, the system then removes everything from the database and after that reads the file that was created to check if the data inserted into the database is inserted correctly

```c
void test_GetHashPerformanceWithNoPatientsPresent(void)
{
    RemoveAllDataFromHashTable();
    CreateHashTable();

    size_t total = 0;
    double average = 0;
    double stddev = 0;

    GetHashPerformance(&total, &average, &stddev);

    TEST_ASSERT_TRUE(total == 0);
    TEST_ASSERT_TRUE(average == 0.0);
    TEST_ASSERT_TRUE(stddev == 0.0);
}

void test_GetHashPerformanceTestManualAddition(void)
{
    RemoveAllDataFromHashTable();
    CreateHashTable();

    size_t total = 0;
    double average = 0;
    double stddev = 0;

    AddPatient("A");
    AddPatient("B");
    AddPatient("C");
    AddPatient("D");

    GetHashPerformance(&total, &average, &stddev);

    TEST_ASSERT_EQUAL_UINT32((uint32_t)4, (uint32_t)total);
    double expectedAvg = (double)4 / (double)HASHTABLE_SIZE;
    TEST_ASSERT_FLOAT_WITHIN(0.015625, (float)expectedAvg, (float)average);
    TEST_ASSERT_TRUE(stddev <= 0.124020 + 0.01);
    TEST_ASSERT_TRUE(stddev >= 0.124020 - 0.01);
    printf("stddev: %f\n", stddev);
}
```

Picture 43: In the picture above the test cases are shown of the GetHashPerformance function. The first test tests the outcomes if there are no patients present and the second test tests the outcomes if there are patients present and prints the standard deviation that was calculated by the GetHashPerformance function.

```
:0:test_GetNumberOfMeasurementsWithTooLongName:PASS
:0:test_GetHashPerformanceNoPatientsPresent:PASS
stddev: 0.124020
```

Picture 44: In the picture above the results of the testcases shown in the previous picture are shown. The print command of the standard deviation is also shown to be printed in the terminal.